

Python OOP

Об'єктно-орієнтоване програмування на Python

Що таке об'єкти?

- Об'єкт — це місце в пам'яті, яке має значення та на яке, можливо, посилається ідентифікатор.
- У Python кожна частина даних, яку ви бачите або з якою контактуєте, представлена об'єктом.

```
>>>str = "This is a String"
```

```
>>>dir(str)
```

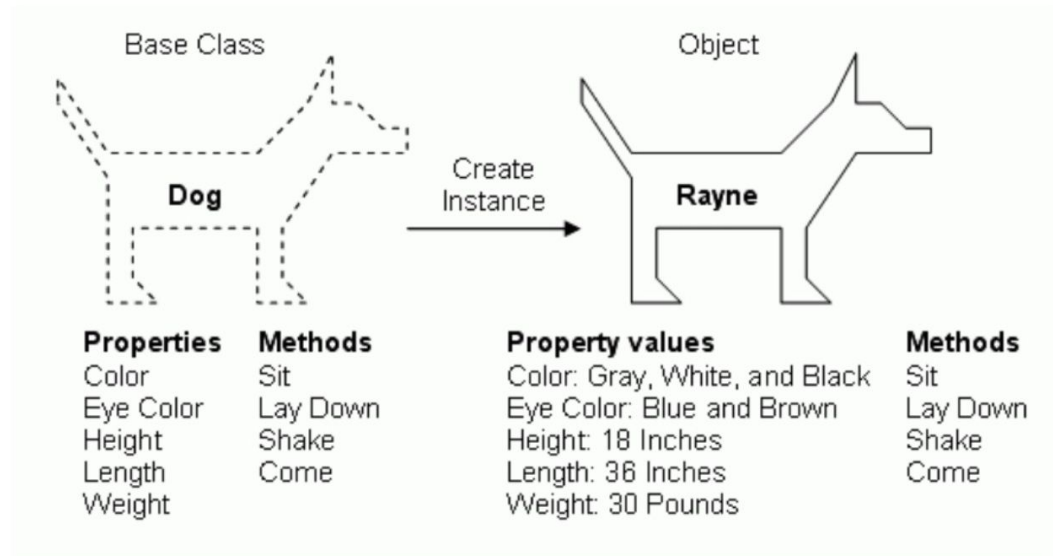
```
.....
```

```
>>>str.lower() 'this is a string'
```

```
>>>str.upper() 'THIS IS A STRING'
```

- Кожен із цих об'єктів має три компоненти:
 - I. Identity
 - II. Type
 - III. Value Arslan Arshad (2K12-BSCS-37)

Визначення класу



Визначення класу

- Механізм класів Python додає класи з мінімумом нового синтаксису та семантики.
- Це суміш механізмів класів, які є в C++ і Modula-3.
- Як C++, члени класу Python є загальнодоступними та мають віртуальні методи.
- Базовий клас складається лише з ключового слова *class*.
- Дайте відповідну назву класу.
- Тепер ви можете створювати члени класу, такі як члени даних і функція-член.

The simplest form of class definition looks like this:

```
class className:  
    <statement 1>  
    <statement 2>  
    .  
    .  
    .  
    <statement N>
```

Визначення класу

- Як ми знаємо, як створити функцію.
- Створіть функцію в класі MyClass під назвою func().
- Збережіть цей файл із розширенням .py
- Ви можете створити об'єкт, викликавши назву класу.
- Python не має нового ключового слова
- Python не має ключового слова new, оскільки все в python є об'єктом.

```
class MyClass:  
    """A simple Example  
of class"""  
    i=12  
    def func(self):  
        return "Hello World"
```

```
>>>obj = MyClass( )  
>>> obj.func()  
'Hello World'
```

Визначення класу

```
1. class Employee:
2.     'Common base class for all employees'
3.     empCount = 0

4.     def __init__(self, name, salary):
5.         self.name = name
6.         self.salary = salary
7.         Employee.empCount += 1

8.     def displayCount(self):
9.         print("Total Employee %d" % Employee.empCount)

10.    def displayEmployee(self):
11.        print("Name : ", self.name, ", Salary: ", self.salary)
```

Визначення класу

- Перший метод `__init__()` — це спеціальний метод, який називається конструктором класу або методом ініціалізації, який Python викликає, коли ви створюєте новий екземпляр цього класу.
- Інші методи класу оголошені як звичайні функції, за винятком того, що першим аргументом кожного методу є *self*.
- *Self*: Це конвенція Python. У слові «self» немає нічого магічного.
- Перший аргумент у `__init__()` та інших функціях gets використовується для посилання на об'єкт екземпляра, і за домовленістю цей аргумент називається *self*. Арслан Аршад (2K12-BSCS-37)

Створення екземплярів об'єктів

"This would create first object of Employee class"

```
emp1 = Employee("Zara", 2000 )
```

"This would create second object of Employee class"

```
emp2 = Employee("Manni", 5000)
```

- Щоб створити екземпляри класу, ви викликаєте клас за допомогою імені класу та передаєте будь-які аргументи, які приймає його метод `__init__`.
- Під час створення екземпляра класу. Python додає аргумент `self` до списку за вас. Вам не потрібно включати його, коли ви викликаєте методи

```
emp1.displayEmployee()  
emp2.displayEmployee()  
print "Total Employee %d" % Employee.empCount
```

Output

```
Name : Zara ,Salary: 2000  
Name : Manni ,Salary: 5000  
Total Employee 2
```


Спеціальні атрибути класу в Python

- За винятком самовизначених атрибутів класу в Python, клас має деякі спеціальні атрибути. Вони забезпечуються об'єктним модулем

Attributes Name	Description
<code>__dict__</code>	Dict variable of class name space
<code>__doc__</code>	Document reference string of class
<code>__name__</code>	Class Name
<code>__module__</code>	Module Name consisting of class
<code>__bases__</code>	The tuple including all the superclasses

Форма та об'єкт для класу

- Клас складається з двох членів: форми та об'єкта.
- Наведений нижче приклад може відобразити різницю між об'єктом і формою для класу.

```
class A:  
    i = 123  
    def __init__(self):  
        self.i = 12345
```

```
print A.i  
print A().i
```

```
>>>  
123  
12345
```

Invoke form: just invoke data or method in the class, so i=123

Invoke object: initialize object Firstly, and then invoke data or Methods.

Here it experienced __init__(), i=12345

Область дії класу

- Ще один важливий аспект класів Python — область видимості.
- Область змінної – це контекст, у якому вона видима програмі.
- Змінні, які доступні всюди (*глобальні змінні*)
- Змінні, які доступні лише членам певного класу (*Змінні-члени*).
- Змінні, які доступні лише для певних екземплярів класу (*змінні екземпляра*)

Знищення об'єктів (вибір сміття)

- Python автоматично видаляє непотрібні об'єкти (вбудовані типи або екземпляри класів), щоб звільнити місце в пам'яті.
- Кількість посилань на об'єкт збільшується, коли йому присвоюється нове ім'я або поміщається в контейнер (список, кортеж або словник).
- Кількість посилань на об'єкт зменшується, коли його видаляють за допомогою `del`, його посилання перепризначають або його посилання виходить за межі області видимості.
- Зазвичай ви не помітите, коли збирач сміття знищить залишений екземпляр і відновить його простір
- Але клас може реалізувати спеціальний метод `__del__()`, який називається деструктором
- Цей метод можна використовувати для очищення будь-яких ресурсів, не пов'язаних із пам'яттю, які використовує примірник

Знищення об'єктів (вибір сміття)

```
a = 40 # Create object <40>  
b = a # Increase ref. count of <40>  
c = [b] # Increase ref. count of <40>  
  
del a # Decrease ref. count of <40>  
b = 100 # Decrease ref. count of <40>  
c[0] = -1 # Decrease ref. count of <40>
```

Спадкування класів

- Замість того, щоб починати з нуля, ви можете створити клас, утворивши його з попередньо існуючого класу, вказавши батьківський клас у дужках після імені нового класу.
- Подібно до підкласу Java може викликати атрибути та методи в суперкласі

Syntax

```
class SubClassName (ParentClass1[, ParentClass2, ...]):  
    'Optional class documentation string'  
    class_suite
```

- Python підтримує множинне успадкування, але ми розглянемо лише успадкування одного батька

```
class Parent: # define parent class
    parentAttr = 100
    def __init__(self):
        print "Calling parent constructor"
    def parentMethod(self):
        print 'Calling parent method'
    def setAttr(self, attr):
        Parent.parentAttr = attr
    def getAttr(self):
        print "Parent attribute :",
        Parent.parentAttr
class Child(Parent): # define child class
    def __init__(self):
        print "Calling child constructor"
    def childMethod(self):
        print 'Calling child method'
```



Static Variable

```
c = Child() # instance of child
c.childMethod() # child calls its method
c.parentMethod() # calls parent's method
c.setAttr(200) # again call parent's method
c.getAttr() # again call parent's method
```

Output

```
Calling child constructor
Calling child method
Calling parent method
Parent attribute : 200
```

Методи перевизначення

- Ви завжди можете перевизначити методи свого батьківського класу.
- Однією з причин перевизначення батьківських методів є те, що вам можуть знадобитися спеціальні або інші функції у вашому підкласі

```
class Parent: # define parent class
    def myMethod(self):
        print 'Calling parent method'
class Child(Parent): # define child class
    def myMethod(self):
        print 'Calling child method'

c = Child() # instance of child
c.myMethod() # child calls overridden method
```


Перевантаження операторів

- У Python ми також перевантажуємо оператори, оскільки там ми перевантажуємо оператор «+».

```
class Vector:
    def __init__(self, a, b):
        self.a = a
        self.b = b
    def __str__(self):
        return 'Vector (%d, %d)' % (self.a, self.b)
    def __add__(self, other):
        return Vector(self.a + other.a, self.b + other.b)

v1 = Vector(2, 10)
v2 = Vector(5, -2)
Print(v1 + v2)
```

Перевантаження методу

- У методі python перевантаження неприйнятне.
- Це буде суперечити духу python, щоб багато турбуватися про те, які типи передаються в методи.
- Хоча ви можете встановити значення за замовчуванням у python, що буде кращим способом.
- Або ви можете:

```
def print_names(names):  
    """Takes a space-delimited string or an iterable"""  
    try:  
        for name in names.split(): # string case  
            print name  
    except AttributeError:  
        for name in names:  
            print name
```

Поліморфізм

- Поліморфізм є важливим визначенням в ООП. Абсолютно, ми можемо реалізувати поліморфізм у Python так само, як і в JAVA. Я називаю це «традиційним поліморфізмом»
- На наступному слайді є приклад поліморфізму в Python.
- Але на Python

Чи існує лише традиційний поліморфізм? Ні

Поліморфізм:

```
class Animal:
    def __init__(self, name): # Constructor of the class
        self.name = name
    def talk(self): # Abstract method, defined by convention only
        raise NotImplementedError("Subclass must implement
                                   abstract method")

class Cat(Animal):
    def talk(self):
        return 'Meow!'

class Dog(Animal):
    def talk(self):
        return 'Woof! Woof!'

animals = [Cat('Missy'), Cat('Mr. Mistoffelees'), Dog('Lassie')]
for animal in animals:
    Print(animal.name + ': ' + animal.talk())
```

У Python скрізь поліморфізм

- Отже, в Python багато операторів мають властивість поліморфізму. Як у наступному

прикладі:

```
>>> 1+2
3
>>> 'key'+'board'
'keyboard'
>>> [1,2,3]+[4,5,6,7]
[1, 2, 3, 4, 5, 6, 7]
>>> (1,2,3)+(4,5,6)
(1, 2, 3, 4, 5, 6)
>>> {A:a, B:b}+{C:c, D:d}
```

- Виглядає безглуздо, але головне в тому, що змінні можуть підтримувати будь-які об'єкти, які підтримують операцію «додавання». Не тільки цілі числа, але й рядки, списки, кортежі та словники можуть реалізувати свою відносну операцію «додавання».

Питання та відповіді